

ODT Complete Reference v3

Technical Concepts Supplement

This supplement adds detailed Technical Concepts sections to every chapter of the ODT Complete Reference. For each source file you will find: the mathematical foundations, embedded systems theory, C programming patterns, and algorithm analysis that you need to understand and extend the code.

XorShift32 Pseudorandom Number Generator

Technical Concept: XorShift32

XorShift32	A linear feedback shift register variant using XOR and barrel shifts. State update: $x^{\wedge}=(x<<13)$; $x^{\wedge}=(x>>17)$; $x^{\wedge}=(x<<5)$. The three constants are chosen so the characteristic polynomial over GF(2) is primitive, giving maximal period $2^{32}-1$.
------------	---

Technical Concept: Period

Period	The generator visits all $2^{32}-1$ non-zero 32-bit states before repeating. The all-zeros state is excluded (absorbing state). Seed=0 is replaced with 1 to avoid this trap.
--------	---

Technical Concept: Rejection sampling

Rejection sampling	Naive modulo bias: if <code>UINT32_MAX</code> is not divisible by bound, values below (<code>UINT32_MAX % bound</code>) appear one extra time. Fix: reject any $x \geq \text{UINT32_MAX} - (\text{UINT32_MAX} \% \text{bound})$. Expected iterations < 2 for any bound.
--------------------	--

Technical Concept: Fisher-Yates

Fisher-Yates	For $i = n-1$ down to 1: swap <code>arr[i]</code> with <code>arr[j]</code> , j uniform in $[0,i]$. Gives exactly $n!$ equally probable permutations. Requires unbiased j selection, which is why <code>rngBounded</code> is used instead of plain modulo.
--------------	--

Technical Concept: Float [0,1)

Float [0,1)	<code>rngNext() >> 8</code> gives 24-bit integer. Dividing by <code>0x1p24f = 2^{24}</code> maps to $[0,1)$. Never returns 1.0. The <code>>> 8</code> discards the 8 least significant bits, which have the worst statistical properties in XorShift.
-------------	---

Bernoulli Sampling and Dropout Theory

Technical Concept: Bernoulli(p)

Bernoulli(p)	Binary random variable: 1 with probability p , 0 with probability $1-p$. Generated by: <code>return rngFloat() < p</code> . Mean = p , Variance = $p(1-p)$, maximised at $p=0.5$.
--------------	---

Technical Concept: Dropout

Dropout	During training, each neuron is zeroed with probability r (dropout rate). Forces neurons to learn independently useful features. At inference, no dropout; weights scaled by $(1-r)$ using inverted dropout.
---------	--

Technical Concept: Inverted dropout

Inverted dropout	Scale kept activations by $1/(1-r)$ during training. This means inference code runs unchanged (no scaling step). Standard dropout scales at inference instead, risking a forgotten scale factor.
------------------	--

Technical Concept: Backward pass

Backward pass	Gradient flows through kept neurons ($\text{mask}=1$) unchanged. Gradient is zeroed for dropped neurons ($\text{mask}=0$). The same mask must be reused in backward as was used in forward -- store the mask tensor.
---------------	--

Technical Concept: Ensemble interpretation

Ensemble interpretation	With n neurons, dropout trains an implicit ensemble of 2^n sub-networks. Each forward pass samples a different sub-network. Averaging their predictions at inference gives regularisation benefit.
-------------------------	--

Preprocessor Macros and Debug Infrastructure

Technical Concept: Variadic macros

Variadic macros	C99 <code>__VA_ARGS__</code> captures ... arguments. GCC extension <code>##_VA_ARGS__</code> removes the preceding comma when zero arguments are passed, allowing <code>PRINT_INFO('msg')</code> with no format args.
-----------------	---

Technical Concept: do-while(false)

do-while(false)	Wraps multi-statement macros: <code>do { stmt1; stmt2; } while(false)</code> . Prevents dangling-else bugs when macro is used in if-else without braces. The compiler optimises away the <code>while(false)</code> check.
-----------------	---

Technical Concept: DLEVEL

DLEVEL	Compile-time constant (0-3) set via <code>-DDEBUG_MODE_ERROR</code> etc. Zero overhead when debug is off: if <code>(0 >= N)</code> is eliminated by the compiler entirely. No runtime branch.
--------	--

Technical Concept: __FILE__ __LINE__

<code>__FILE__</code> <code>__LINE__</code>	Predefined macro tokens: <code>__FILE__</code> expands to the source filename string, <code>__LINE__</code> to the current line number integer, <code>__func__</code> to the function name string. Used to print precise error locations.
---	---

Technical Concept: Token pasting

Token pasting ##	The <code>##</code> operator in macros concatenates two tokens: <code>#define MAKE_ID(x) id_##x</code> expands to <code>id_foo</code> when called with <code>x=foo</code> . Used in ODT to generate unique names per debug level.
------------------	---

Tensor Data Structure and Memory Layout

Technical Concept: Row-major layout

Row-major layout	<code>tensor[i,j,k]</code> at flat index $i \cdot J \cdot K + j \cdot K + k$. All elements of row i come before row $i+1$. This matches C array layout: <code>arr[i][j][k] = *(arr + i * J * K + j * K + k)</code> .
------------------	--

Technical Concept: Zero-copy transpose

Zero-copy transpose	<code>transposeTensor</code> swaps entries in the <code>orderOfDimensions</code> array. The data bytes never move. A matmul that needs W^T just calls <code>transposeTensor</code> , computes, then transposes back -- 0 memcpy cost.
---------------------	---

Technical Concept: BOOL bit-packing

BOOL bit-packing	N booleans stored in $\text{ceil}(N/8)$ bytes. Bit i is at <code>data[i/8]</code> bit position $(i\%8)$. <code>tensorBoolGet</code> : $(\text{data}[i/8] \gg (i\%8)) \& 1$. <code>tensorBoolSet</code> : $\text{data}[i/8] = (1 \ll (i\%8))$ for true, $\&= \sim(1 \ll (i\%8))$ for false.
------------------	--

Technical Concept: shape_t

shape_t	Stores: <code>numberOfDimensions</code> , <code>size_t *dimensions</code> (actual sizes), <code>size_t *orderOfDimensions</code> (logical order, swapped by transpose). <code>orderOfDimensions[0]</code> is the first logical dimension's index into <code>dimensions[]</code> .
---------	---

Technical Concept: parameter_t

parameter_t	Pairs a weight tensor (<code>param</code>) with its gradient tensor (<code>grad</code>). Both have the same shape. The optimizer reads <code>param</code> and <code>grad</code> , updates <code>param</code> in place, then zeros <code>grad</code> for the next step.
-------------	--

Data Type Registry Pattern

Technical Concept: Registry pattern

Registry pattern	A static const array <code>DTYPE_TABLE</code> indexed by <code>qtype_t</code> enum stores size and name per type. All modules call <code>dtypeSize(type)</code> instead of scattering <code>sizeof</code> checks. One source of truth.
------------------	--

Technical Concept: Designated initializers

Designated initializers	C99: <code>[FLOAT32] = {4, 'float32'}</code> places the struct at array index <code>FLOAT32</code> . Sparse initialization: unspecified indices are zero-filled. Allows gaps in enum without bugs.
-------------------------	--

Technical Concept: const in Flash

const in Flash	static const places data in <code>.rodata</code> section, mapped to Flash on STM32. Zero SRAM cost. Flash read costs 1-2 extra cycles via ART Accelerator prefetch, but the SRAM saving is worth it.
----------------	--

Technical Concept: BOOL special case

BOOL special case	BOOL has <code>byteSize=0</code> in the table (no full byte per element). <code>calcBytesPerTensor</code> detects BOOL and uses <code>ceil(n/8)</code> formula instead of <code>byteSize*n</code> . Special-casing is needed because bit packing breaks the standard formula.
-------------------	---

Persistent Storage and Flash/SRAM Layout

Technical Concept: SRAM vs Flash

SRAM vs Flash	STM32F746: 320 KB SRAM (fast, read-write, volatile) and 1 MB Flash (slower, read-only in normal operation, persistent). Model weights live in SRAM during training. Checkpoints are written to Flash or SD card.
---------------	--

Technical Concept: Flash wear leveling

Flash wear leveling	Each Flash sector can be erased ~10,000 times. Writing checkpoints too frequently degrades Flash. Strategy: checkpoint every N epochs, not every step. Use a circular buffer of 2 sectors to double endurance.
---------------------	--

Technical Concept: FatFS

FatFS	Embedded FAT filesystem library. Provides fopen/fread/fwrite compatible API for SD card files. Requires SDIO or SPI driver. Used by NPYLoader and CSVLoader on STM32 to stream training data.
-------	---

Technical Concept: Linker sections

Linker sections	.text (code in Flash), .data (initialised globals in SRAM, copied from Flash at boot), .bss (zero-initialised globals in SRAM), .rodata (constants in Flash). Tensor data buffers go in .bss or are malloc'd from heap.
-----------------	---

Fixed-Point Quantisation Mathematics

Technical Concept: Symmetric quantisation

Symmetric quantisation	$\text{real} = \text{mantissa} * \text{scale}$. $\text{zeroPoint} = 0$. Range: $[-q\text{Max} * \text{scale}, +q\text{Max} * \text{scale}]$. Works well for weights (symmetric distribution). 8-bit SYM: $q\text{Max}=127$, 256 representable values from -127 to 127.
------------------------	--

Technical Concept: Asymmetric quantisation

Asymmetric quantisation	$\text{real} = (\text{mantissa} - \text{zeroPoint}) * \text{scale}$. zeroPoint shifts the range. Handles ReLU activations (always ≥ 0). 8-bit ASYM: can represent $[0, 255 * \text{scale}]$ by setting $\text{zeroPoint}=0$ and using unsigned mantissa.
-------------------------	--

Technical Concept: qMaxBits

qMaxBits	FQT uses $q\text{MaxBits}=12$ for operands ($\text{int}12 \times \text{int}12 = \text{int}24$, fits $\text{int}32$ with headroom). Gradient target uses $q\text{MaxBits}=16$. The bit budget is smaller than 8 for operands to leave accumulation headroom.
----------	--

Technical Concept: Scale calibration

Scale calibration	$\text{scale} = \max(x) / q\text{Max}$. Post-training: calibrate on representative data. FQT: scale is a learned parameter updated by the backward pass. MinMax calibration can be done per-tensor or per-channel.
-------------------	---

Technical Concept: Quantisation error MSE

Quantisation error MSE	For uniform quantiser with step size $\text{delta}=\text{scale}$: $\text{MSE} = \text{delta}^2/12$ assuming uniform error distribution. Reducing scale (smaller step) reduces MSE but narrows the representable range.
------------------------	---

Stochastic Rounding and the Dead Zone Problem

Technical Concept: Dead zone problem

Dead zone problem	If $ \text{gradient} < 0.5 * \text{scale}$, deterministic rounding always gives 0. The weight never updates. For small learning rates and many quantised layers, most weights enter the dead zone and training stalls.
-------------------	--

Technical Concept: Stochastic rounding

Stochastic rounding	SR_HALF_AWAY: $\text{round}(\text{input} + U - 0.5)$ where $U \sim \text{Uniform}[0,1)$. The noise U breaks the dead zone: even a gradient of $0.01 * \text{scale}$ has a 1% chance of rounding up. Expected value: $E[\text{SR}(x)] = x$ exactly.
---------------------	---

Technical Concept: Unbiasedness proof

Unbiasedness proof	For any x in $[n, n+1)$: $x + U - 0.5$ is in $[n-0.5, n+1.5)$. $\text{round}()$ maps it to n with probability $(n+0.5-x)$ and to $n+1$ with probability $(x-n+0.5)$. $E[\text{result}] = n*(n+0.5-x) + (n+1)*(x-n+0.5) = x$. QED.
--------------------	---

Technical Concept: round-half-away-from-zero

round-half-away-from-zero	For values exactly at $x.5$ (e.g. 2.5): round away from zero gives 3 (not 2 as in banker's rounding). Creates slight positive bias for positive numbers but is simpler and matches Python $\text{round}()$ for positive values.
---------------------------	---

Technical Concept: rescaleIntoAccumulatorScale

rescaleIntoAccumulatorScale	Converts a parameter from its own scale to the accumulator scale: $\text{result} = (\text{mantissa} * \text{paramScale} / \text{accScale})$, then rounds. Used in the ExecuteOp epilogue to merge partial sums.
-----------------------------	--

Arithmetic Type Dispatch

Technical Concept: Tagged union dispatch

Tagged union dispatch	<code>arithmetic_t = {arithmeticType_t type, roundingMode_t roundingMode}</code> . The type tag (<code>ARITH_FLOAT32</code> or <code>ARITH_SYM_INT32</code>) selects which kernel to run. No virtual function table, no C++ inheritance.
-----------------------	--

Technical Concept: Float fallback

Float fallback	If a layer has no INT8 kernel, <code>arithmeticFromQuantization()</code> returns <code>ARITH_FLOAT32</code> . <code>ExecuteOp</code> then converts inputs to float, runs the float kernel, converts output back. Correct but slower.
----------------	--

Technical Concept: Rounding as attribute

Rounding as attribute	Different layers may need different rounding modes: weights use <code>SR_HALF_AWAY</code> (stochastic), activations use <code>HALF_AWAY</code> (deterministic). Rounding mode is stored in <code>arithmetic_t</code> , not in the kernel.
-----------------------	---

Technical Concept: Avoiding virtual dispatch

Avoiding virtual dispatch	C++ virtual functions use a vtable (pointer in each object + indirect call). On Cortex-M7, indirect calls cost extra cycles and prevent branch prediction. C function pointers in a static table are faster and the table is in Flash.
---------------------------	--

Four-Phase Operator Execution Framework

Technical Concept: Prologue

Prologue	Before calling the kernel: if any input tensor's dtype != arithmetic dtype, convert it to a stack-allocated scratch buffer of the arithmetic type. This centralises type conversion so kernels only ever see one dtype.
----------	---

Technical Concept: Stack scratch buffer

Stack scratch buffer	alloca() allocates on the stack. On Cortex-M7, the stack pointer (R13/SP) is decremented by the alloca size. Automatically freed when the function returns. No heap fragmentation. Risk: stack overflow if size is too large (max ~4KB safe on 320KB SRAM with typical call depth).
----------------------	---

Technical Concept: Epilogue modes

Epilogue modes	OUT_WRITE: overwrite output tensor. OUT_ACC_DYNAMIC_RESCALE: rescale the arithmetic result to match output quantisation scale, then accumulate. OUT_ACC_FIXED_SCALE: accumulate directly (assumes compatible scales). Dynamic rescale prevents overflow when accumulating multiple operations.
----------------	--

Technical Concept: writesInPlaceSafe

writesInPlaceSafe	If true and input/output are both FLOAT32, ExecuteOp can pass the same pointer to the kernel as both input and output. The kernel reads and writes in place, saving one allocation. Only safe for elementwise ops (no dependency on future input values).
-------------------	---

Technical Concept: opSpec_t

opSpec_t	Specifies: kernel function pointer, context, input tensor list, number of inputs, arithmetic type, output mode, optional auxiliary output, and writesInPlaceSafe flag. A fully self-contained operation descriptor.
----------	---

Matrix Multiplication on ARM Cortex-M7

Technical Concept: Matrix multiply

Matrix multiply	$C[i,j] = \sum_k A[i,k] * B[k,j]$. For an (M x K) matrix times (K x N) matrix: $O(M*K*N)$ multiply-accumulate operations. In ODT linear layer: $Y = X * W^T$, where X is [batch x in_features] and W is [out_features x in_features].
-----------------	---

Technical Concept: Transpose trick

Transpose trick	<code>transposeTensor(W, 0, 1)</code> swaps the logical dimension order. The matmul kernel always computes $C[i,j] = \sum_k A[i,k] * B[k,j]$. With W transposed, $B[k,j] = W[j,k]$ (accessing W's columns as rows) -- no data movement.
-----------------	--

Technical Concept: Cache efficiency

Cache efficiency	Accessing W^T column by column is inefficient in row-major layout (stride = out_features). The transpose trick accesses W row by row (stride = 1). On Cortex-M7 with 4KB L1 data cache, row-by-row access has much better cache hit rate.
------------------	---

Technical Concept: INT8 matmul

INT8 matmul	$\text{int8} \times \text{int8} = \text{int16}$ intermediate. Accumulator must be int32 to safely sum $K=1024$ products: $\text{max product} = 127*127 = 16129$; $1024 * 16129 = 16,515,096 < 2^{31}$. For $K > 133,169$, int32 could overflow -- ODT limits are well below this.
-------------	--

Technical Concept: SIMD on Cortex-M7

SIMD on Cortex-M7	ARM DSP extension: SMLAD instruction multiplies two int16 pairs and accumulates in int32 in a single cycle. For int8 operands, SIMD processes 4 elements per cycle. ODT does not yet use SIMD intrinsics but this is a future optimisation path.
-------------------	--

Threshold Finding for RigL Pruning

Technical Concept: K-th smallest

K-th smallest	<code>findAbsKthSmallestActive</code> : partial selection sort over active weights. K passes of linear scan, each finds the minimum of remaining elements. $O(n \cdot K)$ time, $O(n)$ space. Better than full sort $O(n \log n)$ when K is small ($K \ll n$).
---------------	--

Technical Concept: K-th largest inactive

K-th largest inactive	<code>findAbsKthLargestInactive</code> : same algorithm but on inactive weights, finding maximum. Used to identify the K inactive weights with largest gradient magnitude -- these are grown in the RigL step.
-----------------------	--

Technical Concept: Threshold semantics

Threshold semantics	The K-th smallest value is used as a DROP threshold: drop any active weight with $ w \leq \text{threshold}$. This may drop more than K weights if there are ties at the threshold value. Conservative: err toward more sparsity.
---------------------	--

Technical Concept: Partial sort alternative

Partial sort alternative	<code>std::nth_element</code> (C++ STL) achieves $O(n)$ average using introselect. Since ODT is pure C, this is not available. A quickselect implementation would give $O(n)$ average -- a useful optimisation for large layers.
--------------------------	--

Technical Concept: Memory allocation

Memory allocation	<code>findAbsKthSmallestActive</code> calls <code>malloc(count * sizeof(float))</code> to store active weight magnitudes. On MCU, malloc should be avoided in tight loops. An optimisation: use a pre-allocated scratch buffer passed as parameter.
-------------------	---

Function Pointer Dispatch Tables in C

Technical Concept: Dispatch table

Dispatch table	layerFunctions[] is a static array indexed by layerType_t. Each entry is a struct {forwardFn, backwardFn, calcShapeFn}. Calling a layer dispatches via table[layer.type].forward(layer, input, output). O(1) dispatch, no switch.
----------------	---

Technical Concept: NULL = non-trainable

NULL = non-trainable	If backwardFn == NULL, the layer has no backward pass (e.g. Flatten, which is just a reshape). The training loop checks for NULL before calling backward. This distinguishes inference-only layers from trainable ones.
----------------------	---

Technical Concept: Designated initializers

Designated initializers	C99: [LINEAR] = {linearForward, linearBackward, linearCalcShape}. Unspecified array entries are zero-initialized (NULL function pointers). Adding a new layer type requires only a new enum value and one new table entry.
-------------------------	--

Technical Concept: ABI stability

ABI stability	Function pointer tables are stable across compilation units. The table layout is fixed by the enum values. Adding a new function to the struct (e.g. initFn) requires recompilation of all modules but maintains binary compatibility within a version.
---------------	---

Fully-Connected Layer Mathematics

Technical Concept: Forward pass $Y = XW^T + b$

Forward pass $Y = XW^T + b$

For each batch item: $y = W * x + b$. W is [out x in], x is [in], y is [out]. The transpose of W is used so the matmul reads W 's rows (one per output neuron) dotted with x .

Technical Concept: Weight gradient $dW = dY^T * X$

Weight gradient $dW = dY^T * X$

Chain rule: $dL/dW_{ij} = \sum_b dL/dY_{bi} * X_{bj}$. In matrix form: $dW = dY^T * X$ where dY is the loss gradient w.r.t. the output. This is an outer product sum over the batch.

Technical Concept: Bias gradient $db = \text{sum}(dY)$

Bias gradient $db = \text{sum}(dY)$

$dL/db_i = \text{sum over batch of } dL/dY_{bi}$. Just sum the columns of dY . Cost: $O(\text{batch} * \text{out_features})$ -- much cheaper than weight gradient $O(\text{batch} * \text{out} * \text{in})$.

Technical Concept: Propagated loss $dX = dY * W$

Propagated loss $dX = dY * W$

$dL/dX_{bi} = \sum_j dL/dY_{bj} * W_{ji}$. Passes the gradient backward through the layer to the previous layer's input. Required for all layers before the current one to receive gradients.

Technical Concept: RigL gap

RigL gap

`linearConfig_t` has no `weightMask` field. RigL needs: `tensor_t *weightMask` (BOOL, same shape as weights). The forward/backward passes must zero activations/gradients for masked-out weights. This is the primary missing piece.

1D Convolution for Sequence Modelling

Technical Concept: 1D convolution

1D convolution	$\text{Output}[b, co, t] = \sum_{ci} \sum_k \text{input}[b, ci, t * \text{stride} + k - \text{pad}] * \text{weight}[co, ci, k].$ For HAR: input [B, 9, 128], weight [16, 9, 3], output [B, 16, 128] (with padding=1, stride=1).
----------------	--

Technical Concept: Receptive field

Receptive field	Each output element sees kernel_size input time steps (and all input channels). Stacking two Conv1d layers with kernel=3: receptive field = $3 + 2 * (3 - 1) = 5$ time steps. Deeper networks capture longer temporal patterns.
-----------------	---

Technical Concept: Stride and downsampling

Stride and downsampling	$\text{stride} > 1$ reduces the temporal dimension: $\text{output_length} = (\text{input_length} - \text{kernel} + 2 * \text{pad}) / \text{stride} + 1$. stride=2 halves the sequence, reducing compute for subsequent layers (like a temporal pooling).
-------------------------	---

Technical Concept: Shared weights

Shared weights	The same kernel weights are applied at every time step. This gives: (1) translation equivariance -- the same pattern detected anywhere in the sequence triggers the same output; (2) parameter efficiency -- $16 * 9 * 3 = 432$ parameters for the entire temporal layer.
----------------	---

Technical Concept: Backward: weight grad

Backward: weight grad	$dW[co, ci, k] = \sum_b \sum_t \text{input}[b, ci, t * s + k] * d\text{Output}[b, co, t].$ The weight gradient is a cross-correlation of input with the output gradient.
-----------------------	--

Rectified Linear Unit and Dying ReLU

Technical Concept: ReLU definition

ReLU definition	$f(x) = \max(0, x)$. Gradient: 1 if $x > 0$, 0 if $x < 0$, undefined at $x=0$ (ODT uses 0 by convention). The subgradient at 0 is chosen as 0 for simplicity; other choices (0.5, 1) are also used.
-----------------	--

Technical Concept: Dying ReLU problem

Dying ReLU problem	If a neuron's pre-activation is always negative (due to large negative bias or weight updates), ReLU always outputs 0. Gradient is also 0 forever. The neuron 'dies' and never recovers. Solution: careful initialisation, small learning rate, or use Leaky ReLU.
--------------------	--

Technical Concept: Leaky ReLU

Leaky ReLU	$f(x) = x$ if $x > 0$, αx if $x < 0$ ($\alpha \sim 0.01-0.1$). Always has non-zero gradient, preventing dying. Not implemented in ODT but a useful extension, especially for very deep networks.
------------	---

Technical Concept: Why not sigmoid/tanh

Why not sigmoid/tanh	Sigmoid/tanh saturate for large $ x $, causing vanishing gradients in deep networks. ReLU has gradient = 1 for $x > 0$ regardless of magnitude. This is the key reason deep networks became trainable with ReLU.
----------------------	---

Technical Concept: Forward efficiency

Forward efficiency	ReLU forward: $\max(0, x)$ is one instruction on Cortex-M7 (VMAX.F32 or conditional MOV). Much faster than sigmoid (requires $\exp()$) or tanh. For a 1024-element activation: ~ 1024 cycles vs $\sim 20,000$ for sigmoid.
--------------------	--

Stochastic Regularisation via Masking

Technical Concept: Dropout mask

Dropout mask	Binary mask M with $M_i \sim \text{Bernoulli}(1-r)$. Forward: $y_i = x_i * M_i / (1-r)$. The $1/(1-r)$ scaling (inverted dropout) maintains expected activation magnitude unchanged.
--------------	--

Technical Concept: Training vs inference

Training vs inference	Mask is applied only during training. At inference: no dropout, no scaling needed (because inverted dropout already compensated during training). Controlled by <code>layer.isTraining</code> flag.
-----------------------	---

Technical Concept: Mask storage

Mask storage	The mask tensor is stored in the layer context and reused in the backward pass. Memory: n bits (packed in $\text{ceil}(n/8)$ bytes) per dropout layer per forward pass. For $n=1024$ neurons: 128 bytes.
--------------	--

Technical Concept: Gradient through mask

Gradient through mask	$dL/dx_i = dL/dy_i * M_i / (1-r)$. Gradient is zero for dropped neurons ($\text{mask}=0$) and passes through scaled for kept neurons. This is the straight-through estimator for the binary Bernoulli gate.
-----------------------	--

Technical Concept: Spatial dropout

Spatial dropout	For convolutional layers, drop entire channels rather than individual elements. All time steps of a dropped channel get zero. More aggressive regularisation but maintains spatial structure within kept channels.
-----------------	--

Probability Normalisation and Numerical Stability

Technical Concept: Softmax formula

Softmax formula	$p_i = \exp(z_i) / \sum_j \exp(z_j)$. Output: non-negative, sums to 1 -- a valid probability distribution. The $\exp()$ amplifies differences: if $z_1=5$ and $z_2=3$, ratio $= e^2 \sim 7.4x$.
-----------------	--

Technical Concept: Numerical stability

Numerical stability	Subtract $\max(z)$ before $\exp$: $\exp(z_i - M) / \sum(\exp(z_j - M))$. Same mathematical result (M cancels) but $z_i - M \leq 0$, so $\exp \leq 1$ -- no overflow. Minimum value $\exp(-\text{large})$ underflows to 0 (acceptable).
---------------------	--

Technical Concept: LogSoftmax

LogSoftmax	$\log(\text{softmax}(z))_i = z_i - M - \log(\sum(\exp(z_j - M)))$. Avoids $\exp$ then $\log$ sequence. Cross-entropy loss $= -\log(\text{softmax}(z)_{\text{label}})$, so LogSoftmax + NLL is the numerically stable pair.
------------	--

Technical Concept: Fused gradient

Fused gradient	Softmax + CrossEntropy gradient: $dL/dz_i = p_i - y_i$ where y_i is the one-hot label. The combined gradient is simply subtract 1 from the true class probability. Avoids computing the $n \times n$ Jacobian of softmax.
----------------	---

Technical Concept: Temperature

Temperature	$\text{softmax}(z/T)$: $T < 1$ sharpens the distribution (more confident), $T > 1$ flattens it. Knowledge distillation uses high T (e.g. $T=5$) to transfer soft probabilities from teacher to student network.
-------------	---

Zero-Copy Reshape and Row-Major Layout

Technical Concept: Zero-copy reshape

Zero-copy reshape	Row-major invariant: element at logical index $[n, c, l]$ is at flat offset $n * C * L + c * L + l$. Reshape to $[n, c * L]$: element at $[n, c * L + l]$ is at flat offset $n * C * L + (c * L + l)$. Same flat offset! Shape metadata changes, bytes do not.
-------------------	---

Technical Concept: orderOfDimensions

orderOfDimensions	Flatten updates shape->numberOfDimensions and shape->dimensions. The data pointer is shared (not copied). The original shape is saved in the layer config to restore it in the backward pass.
-------------------	---

Technical Concept: CNN to FC bridge

CNN to FC bridge	Conv1d output $[B, C, L] \rightarrow$ Flatten $\rightarrow [B, C * L] \rightarrow$ Linear. The Linear layer sees a 1D feature vector of size $C * L$ per batch item. All spatial structure is discarded; the Linear must learn to use it.
------------------	---

Technical Concept: Alternative: GAP

Alternative: GAP	Global Average Pooling $[B, C, L] \rightarrow [B, C]$ by averaging over L . Discards temporal position info but is more translation-invariant than flatten. Uses $C * L / C = L$ fewer parameters in the subsequent Linear layer.
------------------	---

Subsampling and Spatial Invariance

Technical Concept: Max pooling

Max pooling	Takes the maximum value within each kernel window. <code>MaxPool1d(kernel=2, stride=2)</code> : every pair of adjacent values is replaced by their maximum. Halves the sequence length. Retains the strongest activation.
-------------	---

Technical Concept: Average pooling

Average pooling	Takes the mean within each window. Smoother than max pooling. Less sensitive to single spikes. Global Average Pooling (window = full sequence) produces a fixed-size output regardless of input length.
-----------------	---

Technical Concept: Translation invariance

Translation invariance	Pooling makes the representation partially invariant to the exact position of a feature. A spike at $t=5$ vs $t=6$ both map to the same output after pooling with $\text{kernel}=2$. Reduces overfitting to exact temporal positions.
------------------------	--

Technical Concept: Backward pass

Backward pass	Max pool backward: gradient flows only to the maximum element (the one that was selected). Other elements get zero gradient. Average pool backward: gradient is uniformly divided among all elements in the window.
---------------	---

Technical Concept: Stride vs overlap

Stride vs overlap	$\text{stride} = \text{kernel_size}$: non-overlapping windows, maximum information reduction. $\text{stride} < \text{kernel_size}$: overlapping windows, smoother output. $\text{stride} > \text{kernel_size}$: gaps between windows, some input not seen.
-------------------	---

Normalisation for Training Stability

Technical Concept: Layer normalisation

Layer normalisation	For each sample independently: compute mean μ and variance σ^2 over features, then $y = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$. Learned scale γ and shift β are applied: $\text{output} = \gamma * y + \beta$.
---------------------	---

Technical Concept: vs Batch norm

vs Batch norm	Batch norm normalises over the batch dimension (requires batch size > 1). LayerNorm normalises over the feature dimension (works with batch size $= 1$ -- critical for MCU with tiny batches). LayerNorm was designed for transformers.
---------------	--

Technical Concept: Epsilon

Epsilon	$\epsilon = 1e-5$ prevents division by zero when variance is very small. Also prevents gradient explosion when the layer input is nearly constant (all values close to the mean, variance near 0).
---------	--

Technical Concept: Backward pass

Backward pass	$dL/dx_i = (1/\sigma) * (dL/dy_i - \text{mean}(dL/dy) - y_i * \text{mean}(dL/dy * y))$. The mean terms subtract the component of the gradient that would change the mean or variance (since normalisation already fixed these).
---------------	--

Technical Concept: Affine transform

Affine transform	γ and β are per-feature learned parameters, initialised to 1 and 0 (identity). They restore the representational power that normalisation removes. Without them, LayerNorm forces a fixed distribution.
------------------	--

Fake Quantisation in Forward Pass

Technical Concept: Fake quantisation

Fake quantisation	During FQT forward pass, activations and weights are quantised then immediately dequantised back to float. The computation still uses float32, but the tensor values have been rounded to representable INT8 values.
-------------------	--

Technical Concept: Why fake

Why fake	True INT8 computation (integer matmul) is used at inference. During training, fake quantisation simulates the rounding error that will occur at inference, so the network learns to be robust to it.
----------	--

Technical Concept: Straight-through estimator

Straight-through estimator	The quantisation <code>round()</code> function has zero gradient almost everywhere. The straight-through estimator (STE) approximates the gradient as 1 through the quantise-dequantise step: $dL/dx_{float} = dL/dx_{quantised}$. Allows gradient flow despite rounding.
----------------------------	--

Technical Concept: Scale learning

Scale learning	FQT learns the quantisation scale alongside the weights. The gradient of the loss w.r.t. scale is computed and scale is updated by the optimizer. This jointly optimises the numeric representation and the computation.
----------------	--

Technical Concept: Deploy vs train

Deploy vs train	At deployment, the fake quantisation is removed and replaced with true integer arithmetic. The float weights are quantised once (using the learned scale) and stored as INT8 in Flash.
-----------------	--

Fractionally-Strided Convolution

Technical Concept: Not the inverse

Not the inverse	Transposed conv does NOT reconstruct the original signal. It produces the output shape that a regular conv would take as INPUT. For a conv with stride 2 that takes $[B, C, L]$ and outputs $[B, C', L/2]$, the transposed conv takes $[B, C', L/2]$ and outputs $[B, C, L]$.
-----------------	---

Technical Concept: Zero insertion

Zero insertion	Stride-s transposed conv = insert $(s-1)$ zeros between input elements, then regular conv. Output length = $(L_{in} - 1) * stride + kernel - 2 * pad$. At stride=2, kernel=4, pad=1: $L_{out} = (L_{in}-1)*2+4-2 = 2*L_{in}$.
----------------	---

Technical Concept: Checkerboard artifacts

Checkerboard artifacts	Stride > 1 creates a periodic zero-insertion pattern. The kernel sees zeros at regular intervals, causing the output to have a grid-like variation at frequency stride. Fix: use resize (linear interpolation) + regular conv instead of transposed conv.
------------------------	---

Technical Concept: Weight gradient

Weight gradient	$dW[ci, co, k] = \sum_b \sum_t input[b, ci, t] * dOutput[b, co, t*stride+k]$. The weight gradient is a cross-correlation of the input with the output gradient, same as regular conv backward.
-----------------	---

Adaptive Moment Estimation with Weight Decay

Technical Concept: Adam update

Adam update	$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ (first moment = EMA of gradient). $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$ (second moment = EMA of squared gradient). $\hat{m}_t = m_t / (1-\beta_1^t)$, $\hat{v}_t = v_t / (1-\beta_2^t)$ (bias correction). $w = w - lr * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$.
-------------	--

Technical Concept: Adaptive rates

Adaptive rates	The update step size for each weight = $lr * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$. For weights with large consistent gradients (large $\hat{m}_t$, small $\hat{v}_t$): larger step. For noisy gradients (large $\hat{v}_t$ relative to $\hat{m}_t$): smaller step. Automatic per-weight learning rate.
----------------	--

Technical Concept: Double precision

Double precision	$\beta_2=0.999$: $1-\beta_2=0.001$. In float32: $1.0f - 0.999f = 0.001007...$ (loses 3 decimal places). In float64: exactly 0.001000... Bias correction ($1-\beta_2^t$) accumulates this error over thousands of steps. ODT stores $\beta_1/\beta_2/\epsilon/\text{weightDecay}$ as double.
------------------	---

Technical Concept: AdamW vs Adam

AdamW vs Adam	L2 weight decay in Adam adds $\lambda * w$ to gradient: $g_{\text{eff}} = g + \lambda * w$. The adaptive scaling then distorts the decay: small-gradient weights get stronger decay, large-gradient weights get weaker decay. AdamW decays weights directly: $w = w * (1 - lr * \lambda)$, then applies Adam update. Decoupled from adaptive scaling.
---------------	---

Technical Concept: Bias correction

Bias correction	$m_0 = v_0 = 0$. At step 1: $m_1 = (1-\beta_1)g_1$, which underestimates the true gradient. Dividing by $(1-\beta_1^1) = (1-\beta_1)$ corrects this: $\hat{m}_1 = g_1$. Correction fades as t grows and $\beta^t \rightarrow 0$.
-----------------	--

Stochastic Gradient Descent and Momentum

Technical Concept: SGD update

SGD update	$w_{t+1} = w_t - lr * g_t$. Simple, fast, no extra memory (no moment tensors). Convergence: for convex losses with $lr < 2/L$ (L = Lipschitz constant), SGD converges. For non-convex neural networks, SGD reaches a stationary point.
------------	--

Technical Concept: Momentum

Momentum	$v_t = momentum * v_{t-1} - lr * g_t$; $w_{t+1} = w_t + v_t$. The velocity v accumulates gradient history. For a consistent gradient direction, effective $lr = lr / (1 - momentum)$. At momentum=0.9: 10x effective learning rate.
----------	--

Technical Concept: MCU memory advantage

MCU memory advantage	AdamW: 2 extra tensors per parameter (first + second moment) = 2x parameter memory. SGD (no momentum): 0 extra. SGD with momentum: 1 extra tensor = 1x parameter memory. For 50K-parameter model: saves 200-400 KB vs AdamW.
----------------------	--

Technical Concept: Nesterov

Nesterov	Gradient computed at $w + momentum * v$ (the 'look-ahead' position) instead of w . Theoretically faster convergence for convex problems. ODT implements classical Polyak momentum for simplicity.
----------	---

Technical Concept: L2 decay in SGD

L2 decay in SGD	$g_{eff} = g + lambda * w$. Update: $w = w - lr * (g + lambda * w) = w * (1 - lr * lambda) - lr * g$. The $w * (1 - lr * lambda)$ term shrinks weights toward zero by a factor $(1 - lr * lambda)$ each step.
-----------------	---

Learning Rate Annealing Strategies

Technical Concept: Step LR

Step LR	Multiply lr by γ every $step_size$ steps: $lr_t = lr_0 * \gamma^{\lfloor t/step_size \rfloor}$. Piecewise constant schedule. Simple to tune. $\gamma=0.1$ reduces lr by 10x every $step_size$ steps.
---------	---

Technical Concept: Cosine annealing

Cosine annealing	$lr_t = lr_{min} + 0.5 * (lr_{max} - lr_{min}) * (1 + \cos(\pi * t / T_{max}))$. Smooth decay from lr_{max} to lr_{min} . Avoids abrupt lr drops that can destabilise training. Widely used for neural networks.
------------------	---

Technical Concept: Exponential decay

Exponential decay	$lr_t = lr_0 * \gamma^t$. Continuous exponential decay. For $\gamma=0.99$ and 1000 steps: $lr_{1000} = lr_0 * 0.99^{1000} = lr_0 * 4.3e-5$. Very aggressive decay -- usually γ is set close to 1 (e.g. 0.999).
-------------------	---

Technical Concept: Warm-up

Warm-up	Start with small lr (e.g. $lr/10$) for the first N steps, then increase to lr_0 . Prevents large initial updates from pushing weights far from their initial values before gradients have stabilised. Not in ODT but a useful extension.
---------	---

Technical Concept: Double precision

Double precision	Schedule scalars (γ^t for exponential, $\cos(\pi * t / T)$) are computed in double to avoid accumulated float rounding error over thousands of steps. Cast to float only at <code>setLr()</code> .
------------------	---

Cross-Entropy and MSE Loss Derivations

Technical Concept: Cross-entropy

Cross-entropy	$L = -\sum_i y_i \cdot \log(p_i)$ where y is one-hot ($y_{\text{label}}=1$, others=0). Simplifies to: $L = -\log(p_{\text{label}})$. Gradient w.r.t. pre-softmax logit z_i : $dL/dz_i = p_i - y_i$ (with fused softmax).
---------------	---

Technical Concept: MSE loss

MSE loss	$L = (1/n) \cdot \sum_i (y_i - \hat{y}_i)^2$. Gradient: $dL/d_{\hat{y}_i} = (2/n) \cdot (\hat{y}_i - y_i)$. Used for regression. For classification, cross-entropy is preferred (matched to softmax, better gradient properties).
----------	---

Technical Concept: REDUCTION_MEAN vs REDUCTION_SUM

REDUCTION_MEAN vs REDUCTION_SUM	Mean divides loss by batch size before backward: gradients are batch-size independent. Sum does not divide: gradients scale linearly with batch size. Mean is preferred -- hyperparameters (lr , λ) do not need re-tuning when batch size changes.
---------------------------------	--

Technical Concept: Numerical stability

Numerical stability	$\log(p)$ where p is near 0 gives $-\infty$. Solution: compute LogSoftmax first (numerically stable), then $\text{NLLLoss} = -\log_{\text{softmax_output}}[\text{label}]$. Avoids computing $p = \exp(\log_p)$ then $\log(p)$.
---------------------	---

Technical Concept: Class imbalance

Class imbalance	Cross-entropy treats all classes equally. For imbalanced datasets (e.g. 90% class A, 10% class B), class B examples rarely contribute to the loss. Fix: weighted cross-entropy ($\text{weight}_i = 1/\text{frequency}_i$). Not in ODT but important for real deployment.
-----------------	--

Mini-Batch Sampling and Shuffling

Technical Concept: Epoch vs batch

Epoch vs batch	One epoch = one complete pass over the training dataset. One batch = a subset of samples processed together before the optimizer step. Gradient is averaged over the batch. Batch size 1: pure SGD. Batch size n: full-batch gradient descent.
----------------	--

Technical Concept: Fisher-Yates shuffling

Fisher-Yates shuffling	At the start of each epoch, <code>rngShuffleIndices()</code> permutes the sample index array. This ensures each epoch sees samples in a different random order, preventing the optimizer from memorising the order.
------------------------	---

Technical Concept: dropLast

dropLast	If the dataset size is not divisible by batch size, the last batch is smaller. <code>dropLast=true</code> : discard the last partial batch. <code>dropLast=false</code> : use it. ODT requires <code>dropLast=true</code> (false path returns an error -- not implemented).
----------	---

Technical Concept: Streaming vs preloading

Streaming vs preloading	Preloading: read entire dataset into SRAM before training. Fast access but requires 30.5MB for HAR -- impossible on 320KB SRAM. Streaming: read one sample at a time from SD card. Slower but fits any dataset. ODT uses streaming.
-------------------------	---

Technical Concept: Sample index array

Sample index array	Store an array of integers <code>[0, 1, 2, ..., N-1]</code> . Shuffle this array each epoch. Iterate through it: load sample indices[i] from storage. This avoids moving sample data during shuffle.
--------------------	--

NumPy Binary Format Parsing

Technical Concept: NPY magic bytes

NPY magic bytes	First 6 bytes: 0x93, 'N', 'U', 'M', 'P', 'Y'. The 0x93 is chosen to detect text files (it is not valid ASCII). If any of these bytes don't match, the file is not a valid .npz file.
-----------------	--

Technical Concept: Version

Version	Byte 6: major version (1 or 2). Byte 7: minor version. v1.0: header length in uint16 (max 65535 byte header). v2.0: header length in uint32 (for very large structured dtypes). HAR files use v1.0.
---------	---

Technical Concept: Header dict

Header dict	ASCII Python dict: {'descr': '<f4', 'fortran_order': False, 'shape': (6617, 9, 128)}.
-------------	---

'<f4' = little-endian float32. 'fortran_order': False = row-major (C order). Shape gives dimensions.

Technical Concept: dtype parsing

dtype parsing	'<f4': < = little-endian, f = floating point, 4 = 4 bytes. '<i4': little-endian int32. '>f4': big-endian float32. ODT only supports '<f4' and '<i4'. STM32 is little-endian so '<' requires no byte-swap.
---------------	---

Technical Concept: Data layout

Data layout	After the header, raw binary data follows immediately. For shape (6617, 9, 128) float32: $6617 * 9 * 128 * 4 = 30,539,776$ bytes. Stored row-major (innermost dimension contiguous). One fread() can load a whole sample.
-------------	---

CSV Parsing with strtok

Technical Concept: CSV structure

CSV structure	Each row is a text line. Fields separated by a delimiter (comma by default). Numbers stored as ASCII: '3.14159' is 7 bytes vs 4 bytes for float32. HAR 6617-row CSV: ~120 MB vs 30.5 MB binary.
---------------	---

Technical Concept: strtok

strtok	Splits a string on a delimiter character. Modifies the string in-place (replaces delimiters with null bytes). Returns pointer to each token sequentially. NOT reentrant (uses hidden static pointer). Use strtok_r for multi-threaded code.
--------	---

Technical Concept: strtod

strtod	Converts ASCII decimal string to float32. Handles: leading whitespace, scientific notation ('1.5e-3'), sign. Reports parse position via endptr. Much slower than direct float32 binary read but robust for human-editable data.
--------	---

Technical Concept: fgets safety

fgets safety	Reads at most n-1 characters plus a null terminator. Prevents buffer overflow. Never use gets() (no length limit, banned in C11). A 4096-byte line buffer handles rows with up to ~500 float values.
--------------	--

ODTS v2 Wire Format and Checkpoint Storage

Technical Concept: ODTS magic

ODTS magic	4-byte magic string 'ODTS' identifies the file format. Enables quick rejection of non-ODTS files without reading further. Common practice in binary formats (ELF: 0x7FELF, PNG: 0x89PNG).
------------	---

Technical Concept: Version field

Version field	uint32 version=2 after the magic. Older readers can check version and reject incompatible files with a clear error. New readers can handle both v1 and v2. Never rely on file extension for format version.
---------------	---

Technical Concept: TLV encoding

TLV encoding	Tag-Length-Value: each record has a u8 tag (identifies the layer type), followed by the payload. Readers skip unknown tags (forward compat). Writers always include known tags (backward compat). Extensible binary protocol.
--------------	---

Technical Concept: serializeSparsity stub

serializeSparsity stub	serializeSparsity() exists in Serialize.c but is an empty stub -- it writes nothing. The sparsity mask is NOT saved in checkpoints. On restart, the mask must be recomputed or reinitialised. This is the key RigL serialisation gap.
------------------------	---

Technical Concept: Endianness

Endianness	Binary protocols must specify byte order. ODTS uses little-endian (same as STM32 and x86). A big-endian host (rare today) would need byte-swapping on read/write. Always document the endianness in the format specification.
------------	---

Elementwise Arithmetic Kernels

Technical Concept: Elementwise ops

Elementwise ops	Operations that apply the same function independently to each element: add, multiply, relu, sigmoid, tanh. $\text{Output}[i] = f(\text{input}[i])$ or $f(\text{input_a}[i], \text{input_b}[i])$. No data dependencies between elements -- trivially parallelisable.
------------------------	--

Technical Concept: SIMD vectorisation

SIMD vectorisation	Cortex-M7 has NEON-like SIMD: processes 4 float32 values per instruction (128-bit SIMD). Elementwise ops can process 4x faster with SIMD. Compiler auto-vectorises if the loop has no aliasing and is aligned.
---------------------------	--

Technical Concept: Broadcasting

Broadcasting	If input shapes differ (e.g. $[B, C, L] + [C, 1]$): the $[C, 1]$ tensor is 'broadcast' to match $[B, C, L]$ by repeating values. ODT checks shape compatibility and handles the index mapping for broadcast dimensions.
---------------------	--

Technical Concept: In-place safety

In-place safety	writesInPlaceSafe in opSpec_t: elementwise ops can read and write the same memory (input and output same pointer) because each element is processed independently. Not safe for ops with data dependencies (convolution, matmul).
------------------------	---

Elastic Weight Consolidation

Technical Concept: Catastrophic forgetting

Catastrophic forgetting	Fine-tuning on new data moves all weights toward the new loss minimum, overwriting weights critical for old tasks. Neural networks have no explicit memory -- all knowledge is implicit in weight values.
-------------------------	---

Technical Concept: EWC penalty

EWC penalty	$L_{\text{total}} = L_B + \lambda/2 \cdot \sum_i F_i \cdot (w_i - w_i^*)^2$. F_i = Fisher information diagonal = $E[(d \log p(y x) / d w_i)^2]$. High F_i means w_i is important for Task A; the quadratic penalty resists changes to it.
-------------	---

Technical Concept: Fisher approximation

Fisher approximation	True Fisher matrix is $n \times n$ (too large). EWC diagonal approximation: F_{ii} only. Computed as average squared gradient over Task A dataset. $O(n)$ storage and compute per parameter -- feasible on MCU.
----------------------	---

Technical Concept: Stability-plasticity

Stability-plasticity	λ controls the tradeoff: high λ = stable (protects Task A, limits Task B learning). Low λ = plastic (learns Task B well, forgets Task A). No universal best λ ; depends on task similarity.
----------------------	---

Technical Concept: RigL synergy

RigL synergy	Inactive (zero) weights are already excluded from EWC. Grown weights (new connections for Task B) do not interfere with Task A active topology. Sparsity naturally separates task representations.
--------------	--

Quantisation Format Conversion

Technical Concept: Symmetric quant

Symmetric quant	$\text{real} = \text{mantissa} * \text{scale}$. $\text{Scale} = \max(\text{real}) / \text{qMax}$. For 8-bit: $\text{qMax} = 127$. Inverse: $\text{mantissa} = \text{round}(\text{real} / \text{scale})$. Clamped to $[-128, 127]$ to prevent overflow in the <code>int8_t</code> cast.
-----------------	--

Technical Concept: Asymmetric quant

Asymmetric quant	$\text{real} = (\text{mantissa} - \text{zeroPoint}) * \text{scale}$. $\text{Scale} = (\text{max} - \text{min}) / (\text{qMax} - \text{qMin})$. $\text{zeroPoint} = \text{round}(\text{qMin} - \text{min}/\text{scale})$. Handles non-zero-centred distributions (e.g. ReLU activations always ≥ 0).
------------------	--

Technical Concept: SYM_INT32 rescale

SYM_INT32 rescale	Accumulator after <code>int8 matmul</code> : $\text{scale} = \text{scale}_x * \text{scale}_w$. Output quantisation $\text{scale} = \text{scale}_{\text{out}}$. Rescale factor = $\text{scale}_x * \text{scale}_w / \text{scale}_{\text{out}}$. Multiply accumulator by this factor, then round and clamp to output range.
-------------------	--

Technical Concept: Stochastic rounding in quant

Stochastic rounding in quant	When converting float to <code>int8</code> : $\text{rounded} = \text{round}(\text{float}/\text{scale} + U - 0.5)$ where $U \sim \text{Uniform}[0,1)$. Breaks the dead zone: small float values ($< 0.5 * \text{scale}$) have a non-zero chance of rounding to ± 1 instead of 0.
------------------------------	--

Technical Concept: Sparse-aware conversion

Sparse-aware conversion	For <code>RigL</code> : inactive weights (<code>mask=0</code>) are always 0.0. Their quantised value is always 0. Skip the multiply and round for inactive weights -- just write 0 directly. Saves compute proportional to sparsity (e.g. 80% faster at 80% sparsity).
-------------------------	--

End-to-End Training Orchestration

Technical Concept: Training loop structure

Training loop structure	For each epoch: shuffle indices. For each batch: load samples, forward pass, compute loss, backward pass, optimizer step, zero gradients. Optionally: RigL step every T iterations.
-------------------------	---

Technical Concept: Gradient zeroing

Gradient zeroing	After each optimizer step, all gradient tensors must be zeroed. Otherwise gradients accumulate across batches (gradient accumulation, sometimes intentional for large effective batch sizes). ODT zeros after each step.
------------------	--

Technical Concept: RigL integration

RigL integration	rigLStep() is called every T steps (e.g. T=100). T is chosen to balance exploration (frequent mask updates) with stability (each mask has enough time to influence the loss). Typical T=100-500 for small networks.
------------------	---

Technical Concept: Validation loop

Validation loop	After each epoch, run inference on the validation set (no gradient computation, no dropout). Compute validation loss and accuracy. Used to detect overfitting (train loss decreasing, val loss increasing) and select best checkpoint.
-----------------	--

Technical Concept: Memory management

Memory management	On MCU, the training loop must not allocate heap memory inside the loop. All buffers (activations, gradients, optimizer states) are allocated once before the loop and reused. Fixed memory footprint throughout training.
-------------------	--

Public API Design for Embedded Libraries

Technical Concept: Opaque pointer pattern

Opaque pointer pattern	UserAPI exposes <code>odt_model_t*</code> as an opaque pointer (forward declaration only, struct definition hidden). Users cannot access internal fields directly. The API functions take and return opaque pointers. Encapsulation without C++.
------------------------	--

Technical Concept: Error codes

Error codes	All public functions return <code>odt_status_t</code> (enum: <code>ODT_OK</code> , <code>ODT_ERR_OOM</code> , <code>ODT_ERR_SHAPE</code> , etc.). Callers must check return codes. On MCU, <code>assert()</code> is often disabled (it calls <code>abort()</code>); explicit error codes are the correct error handling mechanism.
-------------	---

Technical Concept: Fluent initialisation

Fluent initialisation	<code>odt_linear_config_default(&cfg)</code> fills <code>cfg</code> with sensible defaults. User overrides only the fields they care about. Reduces boilerplate and prevents uninitialised struct bugs.
-----------------------	---

Technical Concept: Memory ownership

Memory ownership	API documentation must clearly state who owns each allocation. In ODT: the library allocates tensors; the user owns data buffers passed in. Mismatched ownership causes double-free or use-after-free bugs.
------------------	---

Technical Concept: Versioning

Versioning	<code>ODT_VERSION_MAJOR</code> , <code>ODT_VERSION_MINOR</code> , <code>ODT_VERSION_PATCH</code> in the public header. <code>odt_getVersion()</code> returns the runtime version. Allows the application to check compatibility at startup.
------------	---

Statistical Distributions for Weight Init

Technical Concept: Box-Muller transform

Box-Muller transform	Two independent Uniform[0,1] samples $U_1, U_2 \rightarrow$ two independent $N(0,1)$ samples: $Z_0 = \sqrt{-2 \cdot \ln(U_1)} \cdot \cos(2 \cdot \pi \cdot U_2)$, $Z_1 = \sqrt{-2 \cdot \ln(U_1)} \cdot \sin(2 \cdot \pi \cdot U_2)$. Only sqrtf, logf, sinf, cosf needed -- no rejection sampling.
----------------------	---

Technical Concept: Cached spare sample

Cached spare sample	Box-Muller generates two samples per call. Store the second (Z_1) in a static variable spare. Next call returns spare without recomputing. Halves the average cost: one log+sqrt+sin+cos per two samples.
---------------------	---

Technical Concept: Kaiming (He) init

Kaiming (He) init	For ReLU: $\text{var}(w) = 2/\text{fan_in}$. Uniform: $w \sim U(-\sqrt{6/\text{fan_in}}, \sqrt{6/\text{fan_in}})$. Normal: $w \sim N(0, \sqrt{2/\text{fan_in}})$. Ensures variance of activations stays constant across layers at initialisation.
-------------------	--

Technical Concept: Xavier (Glorot) init

Xavier (Glorot) init	For tanh/sigmoid: $\text{var}(w) = 2/(\text{fan_in} + \text{fan_out})$. Accounts for both input and output fan. Kaiming is better for ReLU because it accounts for the factor-of-2 variance reduction that ReLU introduces.
----------------------	--

Technical Concept: Truncated normal

Truncated normal	$N(0,1)$ values outside $[-2, 2]$ are resampled. Prevents extreme outlier initial weights. Rejection sampling: call normalSample(), if $ x > 2$, call again. Expected 2.3% rejection rate (both tails outside ± 2 sigma).
------------------	---

Dynamic Sparse Training — Full Implementation

Technical Concept: RigL algorithm

RigL algorithm	Every T steps: (1) For each sparse layer, compute drop threshold = K-th smallest active weight . (2) Drop K active weights below threshold (set mask to 0, weight to 0). (3) Grow K inactive weights with largest gradient (set mask to 1, weight to 0). (4) Zero gradients of all inactive weights.
----------------	---

Technical Concept: Cosine K decay

Cosine K decay	$\alpha_t = 0.3 * 0.5 * (1 + \cos(\pi * \text{step} / \text{totalSteps}))$. Starts at 0.3 (30% of active weights updated each T steps), decays to 0 at end. $K_t = \alpha_t * \text{numActive}$. Early training: many mask updates (exploration). Late training: few updates (refinement and stability).
----------------	--

Technical Concept: Drop criterion

Drop criterion	Drop weights with smallest magnitude $ w $. Small magnitude $\Rightarrow$ small contribution to output $\Rightarrow$ likely safe to prune. May drop more than K if multiple weights share the K-th magnitude value (threshold ties).
----------------	---

Technical Concept: Grow criterion

Grow criterion	Grow inactive weights with largest gradient magnitude $ g $. Large gradient $\Rightarrow$ activating this weight would most reduce the current loss. The grown weight starts at 0 so its initial contribution is zero, but the optimizer can immediately move it.
----------------	--

Technical Concept: Zero inactive gradients

Zero inactive gradients	After the RigL step, zero all gradients of inactive weights. Otherwise the Adam optimizer accumulates moment estimates for weights that are not active. When they are later grown, stale moments would cause incorrect updates -- ghost updates.
-------------------------	--

Technical Concept: `serializeSparsity` gap

`serializeSparsity` gap

The sparsity mask (BOOL tensor) is NOT saved in `Serialize.c`. On MCU restart, the mask is lost. To fix: add a mask serialisation section to the ODTS format -- write each layer's mask as a packed BOOL array. The stub in `Serialize.c` is the insertion point.